

Validation-First Modeling and Competition-Side XGBoost Exploration for the Kaggle Spaceship Titanic Benchmark

Dingrong Xue, Fan Xie, and Shijun Cheng

Artificial Intelligence Programme, Department of Computer Science

Faculty of Science and Technology

Beijing Normal-Hong Kong Baptist University (BNBU), Zhuhai, China

Team EAP_Hater@MLW; Corresponding author: Dingrong Xue

u430034083@mail.bnpu.edu.cn

Abstract—This paper reports our final AI3023 course project on the Kaggle *Spaceship Titanic* competition. The task is a binary classification problem on mixed-type tabular data: predict whether a passenger was transported to an alternate dimension using demographics, cabin assignments, spending behavior, and travel-group identifiers. We deliberately separate two goals that are often conflated in competition projects: obtaining a competitive leaderboard result and producing a defensible machine learning study. To do this, we benchmark five machine learning models under a repeated group-aware validation protocol, build an EDA-driven feature space, and separately analyze a competition-side XGBoost branch that produced the team’s highest public score. The clean local benchmark identifies CatBoost as the strongest scientific anchor, while XGBoost offers the best speed-performance compromise and ultimately yields the best public leaderboard result through a more aggressive compact-feature branch. We also show, through shared-audit subgroup disagreement analysis, that the public-best XGBoost branch is specialized rather than uniformly superior to the cleaner CatBoost line. In the public leaderboard snapshot downloaded on May 17, 2026, our team EAP_Hater@MLW ranked 97th out of 2240 teams with a score of 0.81716, while our best clean single-model submission ranked 238th with a score of 0.81061. The main conclusion is that for a highly studied benchmark such as *Spaceship Titanic*, rigorous validation, ablation, and reproducibility are more important to the final analysis than optimizing solely for marginal public-leaderboard gains.

Index Terms—Kaggle, tabular classification, CatBoost, XGBoost, feature engineering, cross-validation, reproducibility

I. INTRODUCTION

Working through the AI3023 course project means entering a Kaggle competition while also submitting a complete machine learning report, packaged code, and slides. Because of this setup, the momentum from quick model changes can clash with deeper analysis. Live rankings encourage fast updates, whereas grading rewards clear methods, repeatable results, valid comparisons, and reasoning behind design choices. For a commonly used benchmark such as *Spaceship Titanic*, score chasing is connected to good modeling practice, but it can still diverge from it in important ways.

This difference shaped how we organized the work. Existing public entries for this competition already include shared code,

small input variations, and methods aimed at climbing the leaderboard. Therefore, a better public score does not automatically reflect stronger science. Instead of treating leaderboard visibility as the only goal, we centered validation as the core of the modeling effort, while still documenting the parallel path that produced the team’s best public outcome.

The paper makes four contributions:

- 1) We construct an EDA-driven feature set from the official competition files, combining raw variables with cabin layout, passenger-group, and expenditure patterns.
- 2) We test five models under a single validation setup and compare both predictive accuracy and computational cost.
- 3) We place repeated group-aware testing before model promotion, with a clear separation between early exploration and later audit decisions.
- 4) We reconstruct the team’s highest-scoring XGBoost branch step by step, so the final public result is connected to our testing path instead of standing alone as a ranking point.

This framing is especially appropriate for *Spaceship Titanic*. With limited size, clear semantic structure, and wide public exposure, patterns in local testing can diverge from public leaderboard behavior. Our final report therefore treats the leaderboard as one evaluation channel, not the only measure of model quality.

II. RELATED WORK

Tree-based methods tend to lead tabular binary-classification tasks, particularly boosted variants such as XGBoost, LightGBM, and CatBoost [2]–[4]. Although they differ internally, all three handle nonlinear feature relationships and heterogeneous feature distributions well. Random Forest provides a useful ensemble-averaging comparison [5], while Logistic Regression remains an important linear reference point [6].

Optuna is frequently used for hyperparameter search in competitive workflows [7]. However, adopting a tuning tool does not automatically make a pipeline academically rigorous.

TABLE I
DATASET SUMMARY AFTER INITIAL INSPECTION

Statistic	Value
Training rows	8,693
Test rows	4,277
Raw predictive columns	13
Positive class rate	50.36%
Largest missing rate	2.50%
Median age	27
Zero-spend rate	42.02%
Multi-passenger rate	44.73%
Group-homogeneous rate	87.18%

Greater importance lies in whether the optimization objective, cross-validation approach, and criteria for accepting a change are defined consistently. In a wider context, leaderboard practice has been studied through the lens of reliability: Blum and Hardt show that repeated reliance on public feedback can distort model comparison when public score becomes the only guide for decisions [8]. This insight applies naturally to our class project.

Open Kaggle notebooks also helped us identify practical strategies relevant to this challenge. Representative examples include Cortinhas’ full walkthrough and Arunklenin’s exploratory-analysis and feature-engineering notebook [9], [10]. Common threads appear across these sources: cabin decomposition adds value, passenger-group structure matters, and total expenditure is often more stable than isolated ancillary-spend variables. These shared resources served as background inspiration during early exploration, but they did not replace our own validation and ablation checks.

III. DATASET AND EXPLORATORY ANALYSIS

A. Dataset Characteristics

The official competition dataset contains 8,693 labeled training rows and 4,277 unlabeled test rows [1]. Thirteen raw predictive columns form the initial structure, covering demographics, destination, cryosleep status, cabin code, five spending categories, and personal names. The target variable Transported is nearly balanced, with a positive-class rate of 50.36%. Missingness is mild; no raw feature exceeds a 2.50% missing rate.

Not every row should be treated as an independent draw. Each PassengerId follows the pattern gggg_pp, where the prefix links passengers traveling in the same group. In our summary, 44.73% of rows belong to multi-passenger groups. Because these ties are strong, ignoring them during validation would make the evaluation less credible.

Table I summarizes the core statistics used throughout the paper.

B. Exploratory Data Analysis

Although EDA shaped the modeling work, it was not included merely to fill space. Figure 1 confirms two global properties: missing values are few but consistent, and the target

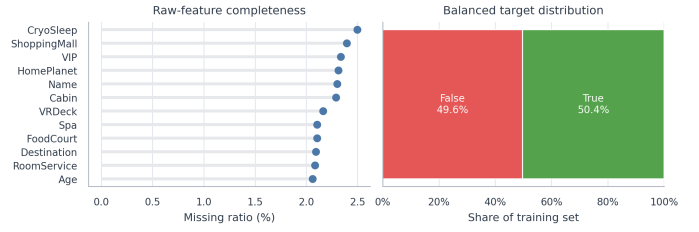


Fig. 1. Data completeness and target balance. The dataset is not sparse, but careful imputation remains necessary.

classes are almost evenly split. This balance makes accuracy a reasonable primary evaluation measure, while F1 and ROC-AUC remain useful for examining finer prediction differences.

Figure 2 reveals less obvious trends. Age contributes signal nonlinearly, particularly when separating children from adults. Spending is more discriminative: transported passengers are strongly concentrated near zero ancillary spend, which matches the interpretation of cryosleep behavior. Large differences also appear across home planets and cabin decks, motivating both category-aware models and deck-based engineered features.

Figure 3 adds two further design cues. Individual travelers form the largest segment, yet shared bookings remain common enough to justify models that recognize group dynamics. The spending variables are correlated with total expenditure but not perfectly overlapping, so the data does not compress neatly into one financial metric. This supports preserving both aggregate and component-level spend information.

IV. METHODOLOGY

A. Preprocessing and Feature Engineering

One guideline shaped the preprocessing: every crafted feature should stand on its own using only information from the official competition files, and it should be clear enough to explain when questioned. The final feature space consists of five families:

- 1) Original demographics and travel descriptors, including HomePlanet, Destination, Age, VIP, and CryoSleep.
- 2) Cabin characteristics, including Deck, CabinNum, Side, and deck-side combinations derived from cabin codes.
- 3) Passenger-group-derived features, including PassengerGroup, PassengerNo, GroupSize, and IsAlone.
- 4) Spending-pattern features, including TotalSpend, zero-spend flags, spend-composition ratios, and log-transformed spend.
- 5) Rule-consistency features, especially indicators that connect cryosleep status with spending behavior.

Table II summarizes these families and the intuition behind them.

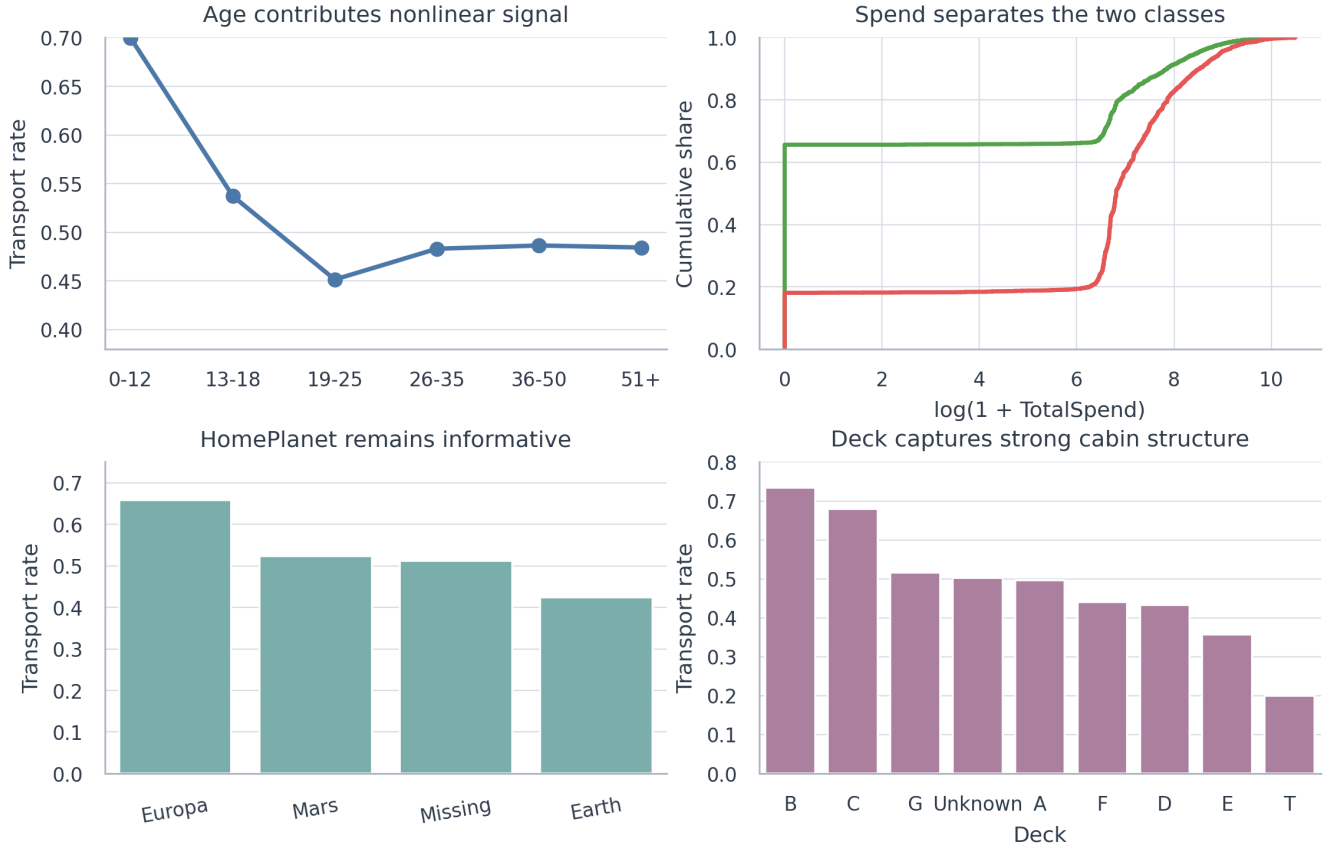


Fig. 2. Core EDA findings. Spending, home planet, and deck have clear predictive value, while age contributes weaker but still meaningful nonlinear structure.

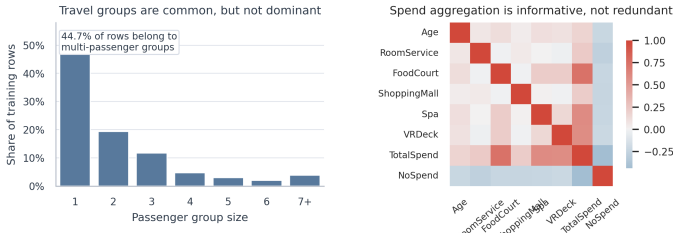


Fig. 3. Passenger-group structure and numeric correlation. These patterns motivate group-aware validation and spend aggregation instead of raw-column-only modeling.

B. Validation Protocol

Validation design was the key methodological decision. Because passengers in the same group can share outcomes, row-wise random splits risk exposing relational structure across folds. We therefore used repeated `StratifiedGroupKFold`, grouping on the `PassengerId` prefix to preserve independence between splits.

Different seeds were assigned different roles:

- Search seeds $\{7, 21, 42, 87, 123\}$ were used for exploratory tuning and feature experiments.
- Audit seeds $\{3, 11, 17, 29, 57, 68, 99, 131\}$ were reserved for promotion decisions.

TABLE II
ENGINEERED FEATURE FAMILIES

Family	Rationale and examples
Original demographics	HomePlanet, Destination, Age, VIP, CryoSleep. These preserve the official schema.
Cabin-derived	Deck, CabinNum, Side, and DeckSide. These encode spatial structure inside the ship.
Passenger-group-derived	PassengerGroup, PassengerNo, GroupSize, and IsAlone. These model travel-group behavior explicitly.
Spend-derived	TotalSpend, NoSpend, spend ratios, and LogTotalSpend. These summarize luxury-service behavior more cleanly.
Rule-consistency	Cryosleep-spend consistency flags. These encode plausible behavioral constraints.

The promotion rule was intentionally conservative. A candidate needed to improve out-of-fold accuracy by at least 0.001 on both seed groups while keeping the audit threshold fixed at 0.5. Because of this rule, several attractive changes were not promoted when their gains failed to remain consistent.

C. Model Families and Tuning Strategy

The course requires at least four models, so we benchmarked five: Logistic Regression, Random Forest, LightGBM, XGBoost, and CatBoost. Logistic Regression provides a linear reference point, while Random Forest acts as a bagging

TABLE III
COMPETITION-SIDE XGBOOST BRANCH CONFIGURATION

Component	Setting
Categorical harmonization	Train and test concatenated before categorical imputation and one-hot alignment
Imputation logic	Mean imputation for numeric columns, most-frequent imputation for compact categorical set, plus room-guided filling for VIP, Cabin, HomePlanet, and Destination
Compact branch features	27 encoded features before pruning; 15 after pruning
Outlier diagnostic	IsolationForest with contamination = 0.003 was evaluated during branch reconstruction but not used in the final submitted training matrix
Class balancing	KMeansSMOTE; training rows increased to 8,762
Final XGBoost setting	seed 36086; max_depth=5, n_estimators=950, learning_rate=0.0475, subsample=0.96, colsample_bytree=0.93, reg_alpha=4.582, reg_lambda=3.06

counterpart. LightGBM, XGBoost, and CatBoost represent the dominant boosting family for tabular competitions.

The tuning strategy depended on what each model was meant to do. In the validation-first setup, all models worked within the same feature space and shared repeated group-aware splits, so comparisons remained fair. LightGBM and CatBoost received controlled local adjustments near already strong starting points. XGBoost appeared in two different roles:

- 1) as part of the clean unified benchmark;
- 2) as a separate competition-side branch focused on improving public leaderboard performance.

Although it was built from the same raw data, the competition-side XGBoost branch relaxes some cleanliness constraints used in the standard benchmark. It concatenates training and test data during categorical harmonization, fills missing values with room-level guidance, reduces the encoded feature space from 27 to 15 dimensions through manual pruning, balances the training matrix using KMeansSMOTE, and then trains a regularized XGBoost model. The best public outcome used random seed 36086, max_depth=5, n_estimators=950, learning_rate=0.0475, subsample=0.96, colsample_bytree=0.93, reg_alpha=4.582, and reg_lambda=3.06. This path is reported separately because its goal was leaderboard performance, not because it is the cleanest scientific baseline.

D. Experimental Controls and Reproducibility

Because defensibility mattered, we maintained several controls across the validation-first benchmark. All unified model comparisons used the same feature families, the same group-aware split design, the same fixed threshold on audit seeds, and no external prediction files. As a result, differences in the benchmark table reflect model-family behavior rather than hidden changes in data handling.

We also kept separate tuning evidence for the clean benchmark models. The LightGBM random-search winner improved

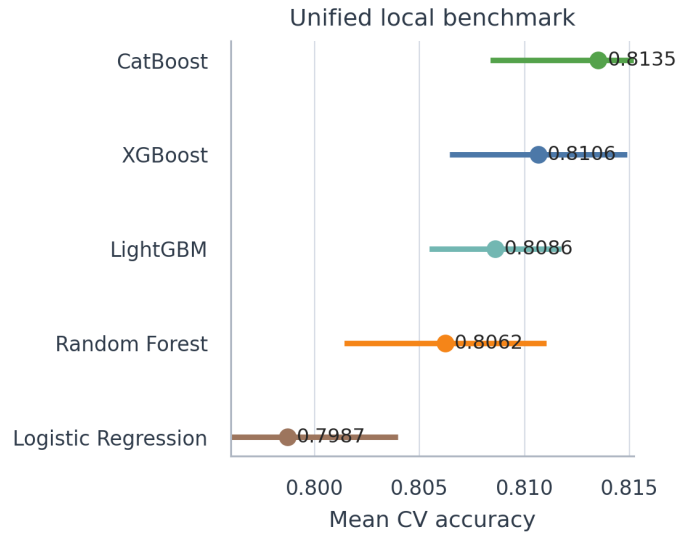


Fig. 4. Mean CV accuracy under the unified benchmark. Boosting clearly dominates the task.

audit accuracy by +0.0031 over the default baseline, while the best CatBoost sweep improved audit accuracy by +0.0023. These gains are reported explicitly later because the assignment requires visible proof of tuning work rather than unsupported claims.

The competition-side XGBoost branch was deliberately kept apart from this evaluation. Its purpose was different: to maximize public leaderboard score within the official competition rules. Separating the tracks helped avoid a common issue in class projects, where a score-oriented entry gradually replaces the more transparent benchmark without being clearly identified as a different object.

V. EXPERIMENTAL STUDY AND RESULTS

A. Unified Local Benchmark

Table IV reports mean CV accuracy, OOF F1, OOF AUC, training time, inference cost, and artifact size under the same benchmark scaffold. Meeting the course criterion requires weighing results not only by predictive score, but also by practical usability.

Figure 4 gives a visual overview of the rankings. CatBoost performs best in the local validation setup, while XGBoost follows closely and trains much more quickly. The linear baseline trails behind, indicating that simple linear assumptions do not capture the structure of the task.

B. Why the Models Behave Differently

The benchmark is informative not only for ranking models, but also for explaining how each model fits the data. Logistic Regression is weakest because the task contains interacting nonlinear effects: cryosleep shapes spending behavior, group travel links tightly with cabin placement, and raw cabin codes become useful only after being unpacked. Random Forest improves on the linear baseline by modeling such interactions,

TABLE IV
UNIFIED LOCAL BENCHMARK UNDER THE SAME FEATURE SET AND VALIDATION REGIME

Model	Mean CV Acc.	OOF F1	OOF AUC	CV Fit Time (s)	Test Infer. (ms/sample)	Artifact Size (MB)
CatBoost	0.8135	0.8143	0.9069	288.32	0.0230	1.37
XGBoost	0.8106	0.8103	0.9034	3.19	0.0061	1.04
LightGBM	0.8086	0.8099	0.9012	5.20	0.0089	1.65
Random Forest	0.8062	0.8013	0.8930	4.79	0.0350	62.40
Logistic Regression	0.7987	0.8047	0.8887	0.46	0.0040	0.01

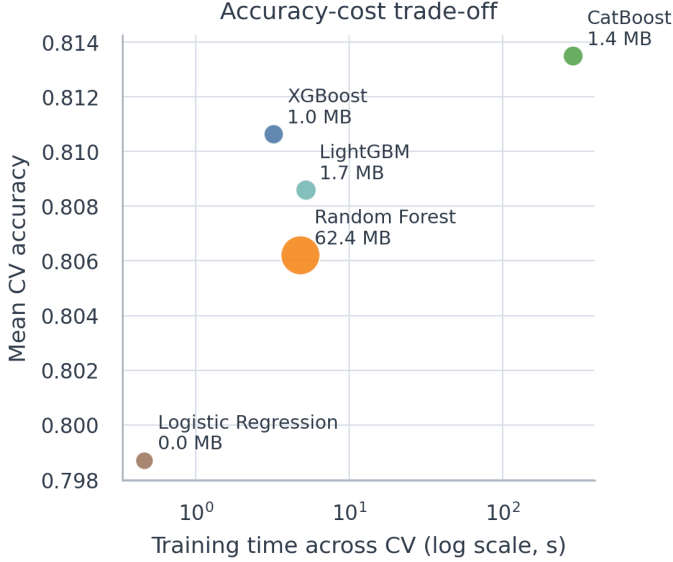


Fig. 5. Accuracy-cost trade-off. CatBoost is strongest locally, while XGBoost is the most attractive speed-performance compromise.

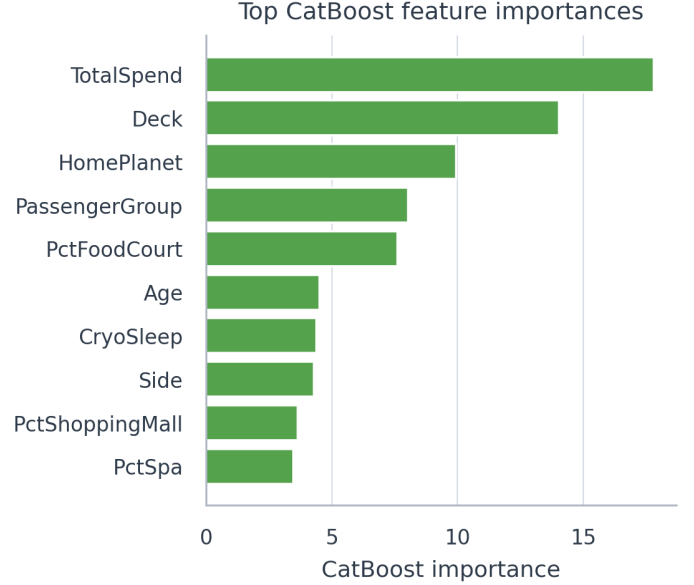


Fig. 6. Top feature importances from the clean CatBoost anchor. The strongest signals align well with the EDA findings.

but it consumes more resources while still falling short of the boosted methods.

LightGBM and XGBoost fit the task more naturally because they handle complex feature relationships quickly on encoded tabular data. CatBoost performs best locally because the dataset mixes categorical and numerical structure in a way that matches its design. This agrees with the EDA: home planet, cabin deck, side, group, and expenditure patterns combine in nuanced ways that go beyond simple numeric effects, yet remain well suited to traditional boosted tree models.

Table V summarizes these qualitative fit arguments.

C. Interpretability of the Clean CatBoost Anchor

Among the tested models, CatBoost is also the easiest anchor to interpret. Figure 6 shows the top feature importances extracted from the validation-first CatBoost benchmark. TotalSpend, Deck, HomePlanet, and PassengerGroup dominate the ranking. These top indicators match patterns observed during EDA, but the model makes them sharper and more concrete. This is why the clean CatBoost pipeline works well as the report centerpiece: its main signals are meaningful in context instead of depending on opaque post-processing.

D. Feature-Family Ablation

Starting from the official inputs, we added feature families step by step to check which groups actually moved performance. The evaluation used the clean CatBoost line under the same group-aware validation setup. This traced gains back to their source instead of assuming that every plausible feature was useful. Figure 7 summarizes the result.

The pattern is more informative than a simple monotonic feature-accumulation story. Raw official features alone reached 0.7973. Adding cabin decomposition raised group-aware CV to 0.8125, the largest jump in the study. Adding group features changed the result only slightly, while spend-summary and rule-consistency features provided no further gain under this fixed CatBoost setup and slightly reduced the final mean. This does not mean that group or spending information lacks value. Rather, in this specific CatBoost configuration, later feature families were partly redundant with the already strong cabin-enhanced representation. The result supports cautious feature selection instead of unchecked inclusion.

E. Tuning Evidence on Clean Models

Figure 8 summarizes the audited gains from the two clean-model tuning tracks retained for the final analysis.

TABLE V
QUALITATIVE SUITABILITY OF EACH BENCHMARKED MODEL FOR THIS DATASET

Model	Main strengths on this task	Main weaknesses on this task	Overall suitability
Logistic Regression	Very fast, interpretable, and useful as a sanity-check baseline.	Cannot capture nonlinear cryosleep-spend-group interactions compactly.	Low. Good baseline, weak final model.
Random Forest	Flexible nonlinear baseline with few distributional assumptions.	Large artifact size and weaker score than boosting under the same feature space.	Moderate. Reasonable baseline, not the best trade-off here.
LightGBM	Fast training and strong performance on sparse encoded features.	Small local gains proved unstable under wider audit checks.	High. Strong clean benchmark model.
XGBoost	Best speed-performance compromise and highly competitive competition model.	Sensitive to preprocessing and feature-pruning choices.	High. Strong practical model, especially for the competition-side branch.
CatBoost	Best local accuracy and very natural fit for mixed-type tabular data.	Slower to train; not the top public-score branch.	Very high. Best scientific anchor for the final report.

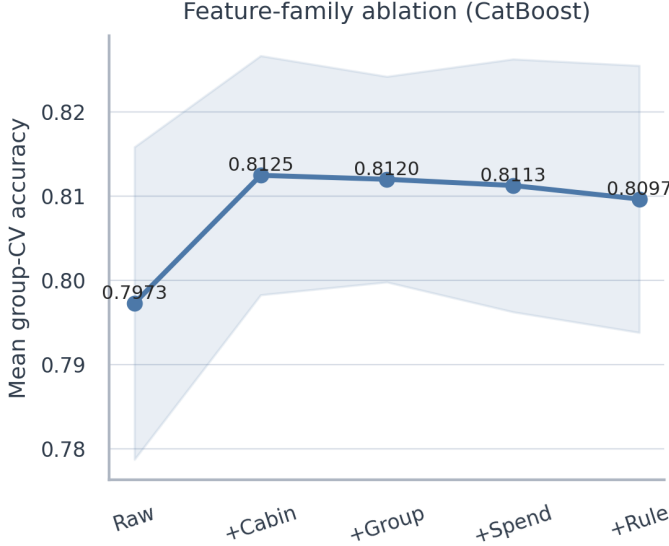


Fig. 7. Cumulative feature-family ablation on the clean CatBoost line. Cabin decomposition produced the largest single improvement.

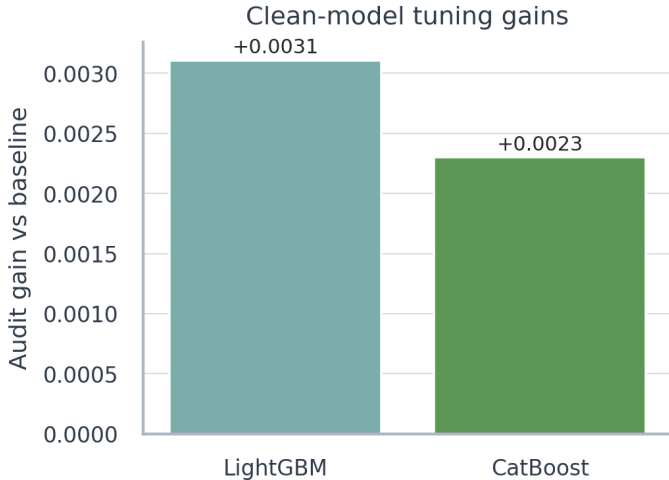


Fig. 8. Audited tuning gains for the clean LightGBM and CatBoost search tracks retained for the final analysis.

The LightGBM track produced a small but verified audit gain of +0.0031 through random search with independent audit

seeds. The CatBoost sweep produced a +0.0023 gain through a narrower search near already strong settings. These changes are subtle, but they satisfy the course requirement for visible hyperparameter tuning while staying within standard reporting limits.

Table VI makes the tuning path more concrete. The winning LightGBM candidate became smaller in leaf capacity, deeper in explicit tree depth, and more conservative through stronger regularization and larger minimum leaf size. The winning CatBoost candidate moved in a similar direction: slightly deeper trees, stronger L_2 regularization, modest subsampling, and less aggressive feature sampling noise. In other words, the preserved search evidence points to disciplined regularization and capacity control rather than to exotic hyperparameter tricks.

F. Validation Alignment versus Public Leaderboard

One central finding is that local validation scores and public leaderboard results do not align monotonically. Figure 9 shows this directly. A high-CV CatBoost candidate reached 0.81905 under local group-aware validation but only 0.80593 publicly. In contrast, the team’s public-best XGBoost branch reached only 0.80516 under the strict group-aware audit but scored 0.81716 on the public leaderboard. These cases show how internal metrics and external ranking behavior can diverge sharply near the top of a heavily examined competition.

This mismatch changed how we worked. We stopped trusting small local gains unless they survived the audit seeds. We also separated the best clean single model from the best public submission instead of forcing one narrative to explain both. The clean CatBoost pipeline remains the most defensible report anchor and scored 0.81061 publicly. The best public result belongs to a different XGBoost branch with a different objective.

Table VII summarizes the candidate lines that mattered most to this conclusion. The table is useful because it separates three different concepts that are easy to blur together in a Kaggle report: the strongest clean scientific anchor, the strongest local-CV counterexample, and the strongest public-leaderboard branch.

TABLE VI
KEY HYPERPARAMETER CHANGES RETAINED FROM THE CLEAN TUNING RECORDS

Model	Baseline setting	Tuned winner setting	Audit gain
LightGBM	500 trees, 31 leaves, unlimited depth, subsample=0.90, colsample=0.80, reg_alpha=0.10, min_child_samples=25	1000 trees, 15 leaves, depth 6, full row/column sampling, reg_alpha=0.70, min_child_samples=35	+0.0031
CatBoost	depth 6, l2_leaf_reg=3, subsample=0.80, rsm=1.00, random_strength=1, min_data_in_leaf=1	depth 7, l2_leaf_reg=7, subsample=0.88, rsm=0.85, random_strength=0.5, min_data_in_leaf=8	+0.0023

TABLE VII
REPRESENTATIVE CANDIDATE LINES AND WHAT THEY TAUGHT US

Candidate line	Local metric	Public score	Role in the project	Main lesson
LightGBM baseline	0.81226	0.80640	Early validation-first baseline	Useful baseline for promotion decisions, but not strong enough as a final competition submission.
Clean CatBoost single model	0.81330	0.81061	Best clean single-model anchor	The most defensible report centerpiece: strong score, simple preprocessing, and high interpretability.
Clean CatBoost reproduction	0.81422	0.80991	Transparent clean reproduction	A respectable public reproduction that still failed to beat the clean anchor.
High-CV CatBoost candidate	0.81905	0.80593	High-CV counterexample	Strongest evidence that local group-aware CV alone is not a dependable public-score ranker near the top of this competition.
Public-best XGB branch	0.80516	0.81716	Best leaderboard branch	Highest public result, but not the strongest scientific baseline; must be explained as a separate competition-side line.

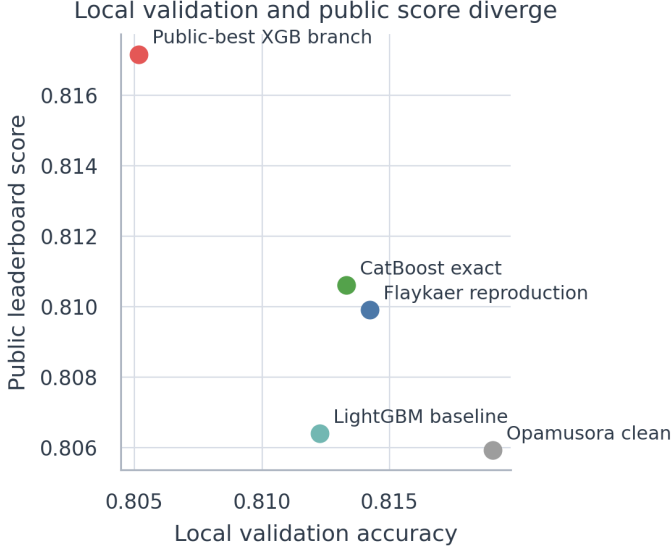


Fig. 9. Local validation does not monotonically predict public leaderboard score. This is why the report emphasizes validation design and ablation rather than public-score optimization alone.

G. The XGBoost Exploration Branch

The highest public score in the team history did not appear from one isolated attempt. It emerged through an XGBoost-centered exploration line. Figure 10 shows this trajectory. An early single XGBoost submission reached 0.79378 on April 1, 2026; a revised single-model version reached 0.80383 on April 10, 2026; an XGBoost-assisted stack reached 0.81084 on April 16, 2026; and the final public-best XGBoost branch reached 0.81716 on April 26, 2026.

This distinction matters because the final XGBoost settings are not claimed to be a unique local-CV optimum. The more

defensible claim is narrower: the public-best branch has visible experimental provenance. It began from an Optuna-guided compact XGBoost setup. Preprocessing, manual feature pruning, and KMeansSMOTE were then kept as fixed branch decisions. Later checks adjusted tree count, learning rate, and seed. The final public-best setting used 950 trees, a learning rate of 0.0475, and seed 36086 while retaining the earlier regularization and sampling choices. A local neighborhood search showed that this result sits within a cluster of strong alternatives rather than at an isolated peak. Therefore, the public-best submission is best read as an evidence-backed competition artifact, not as an unsupported parameter choice.

To understand the success of the final branch, we rebuilt it step by step and measured how its local behavior changed. Table VIII reports the result. The compact preprocessing branch reached 0.80445 under 10-fold row-level reconstruction CV. The IsolationForest diagnostic made this reconstruction metric worse when treated as a filtering step, while manual feature pruning raised it to 0.80755. Re-evaluated under our stricter 5-fold StratifiedGroupKFold audit, the same branch scored 0.80516.

Table IX makes the public-side evolution explicit. The rejected tuned XGBoost variant on April 10, 2026 is especially informative: parameter changes alone did not help. Progress emerged only after the team tested XGBoost within a wider set of competition-oriented model choices rather than pushing it as a direct standalone upgrade.

We also ran a focused local neighborhood search around the branch parameters to test whether the public-best configuration was at least locally competitive under standard validation. Figure 11 shows the result. Row-level CV slightly preferred a lower learning rate with more trees, while the group-aware audit responded better to leaner sampling. However, all tested variants stayed within a narrow accuracy band, and no option

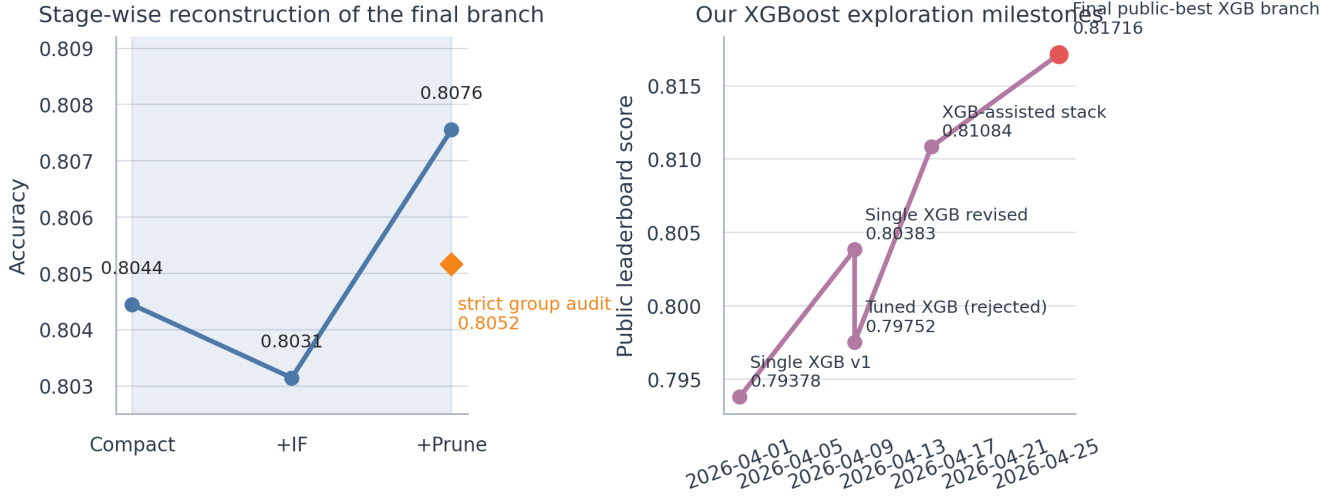


Fig. 10. Left: stage-wise reconstruction of the final XGBoost branch. Right: the public-score trajectory of our XGBoost-centered exploration path from April 1, 2026 to April 26, 2026.

TABLE VIII
STAGE-WISE RECONSTRUCTION OF THE FINAL XGBOOST BRANCH

Stage	Validation view	Accuracy	Std. dev.	Interpretation
Compact preprocessing branch	10-fold row-level reconstruction CV	0.80445	0.01417	A workable but not yet competitive compact XGBoost baseline.
+ IsolationForest diagnostic	10-fold row-level reconstruction CV	0.80314	0.01590	Outlier filtering did not help locally and was not used in the final submitted training matrix.
+ Feature pruning	10-fold row-level reconstruction CV	0.80755	0.01542	Manual pruning recovered a clearer gain and created the final compact branch used for the public-best submission.
Strict group-aware audit	5-fold StratifiedGroupKFold	0.80516	0.00490	Under the stricter report-standard audit, the branch remains weaker than the clean CatBoost anchor despite its public success.

TABLE IX
MILESTONES IN THE XGBOOST EXPLORATION BRANCH

Date	Milestone	Public score	Why it mattered
2026-04-01	Single XGB v1	0.79378	Established an early XGBoost baseline and showed that a straightforward single-model configuration was not yet competitive.
2026-04-10	Single XGB revised	0.80383	Confirmed that XGBoost could improve materially with better preprocessing, even before branch-level redesign.
2026-04-10	Tuned XGB (rejected)	0.79752	Demonstrated that naive tuning did not help and that parameter changes alone were insufficient.
2026-04-16	XGB-assisted stack	0.81084	Showed that XGBoost became useful as a controlled component inside a stronger blended system.
2026-04-26	Final public-best XGB branch	0.81716	Produced the highest public score in the team history and finalized the competition-side narrative.

clearly dominated under both evaluation views.

This finding shapes the logic of the paper. The top public XGBoost variant should not be called the best local solution. A more precise interpretation is that it lies inside a cluster of similarly strong models, with its public edge coming from interactions with the leaderboard split that local CV does not fully capture. This framing is more defensible than asserting a tuning success beyond what the evidence shows.

Two points follow. First, the final branch is not a standalone value floating apart from the project; it is the endpoint of a longer XGBoost exploration line in our own submission history. Second, the branch is best interpreted as a competition-side artifact rather than as the strongest scientific baseline. Its

public success is real, but the strongest methodological claim still comes from the validation-first benchmark.

H. Where the Two Branches Disagree

The previous subsection explains why the public-best XGBoost branch should not be described as a universal local optimum. A complementary question is whether it behaves like a uniformly stronger classifier than the cleaner CatBoost line. Under one shared 5-fold StratifiedGroupKFold audit, the answer is no. The two branches disagree on 8.49% of training rows, and the cleaner CatBoost line keeps a 0.37-point accuracy advantage overall under that shared audit.

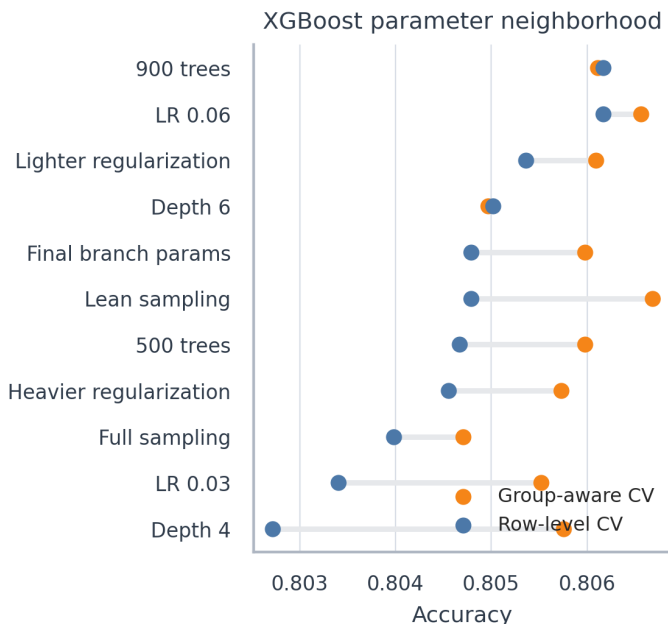


Fig. 11. Focused local neighborhood search around the final XGBoost branch. The public-best configuration sits inside a competitive local plateau rather than as a dominant local optimum.

Figure 12 shows where that disagreement concentrates. At the cabin level, the two branches diverge most strongly on decks E and G. At the behavioral level, disagreement is highest in cryosleep-spend edge cases, especially passengers with missing `CryoSleep` but zero spend and passengers marked as `CryoSleep=False` despite also having zero spend. These are exactly the ambiguous cases where heuristic competition-side preprocessing can shift predictions. XGBoost is not uniformly worse: it performs slightly better on narrower subsets such as Deck A and Deck E. The right interpretation is therefore specialization. The compact XGBoost branch changes a small but meaningful subset of decisions, some of which help publicly, but it does not dominate the cleaner CatBoost line across the board.

I. Final Kaggle Ranking and Submission History

The public leaderboard snapshot downloaded on May 17, 2026 places our team `EAP_Hater@MLW` at Rank 97 out of 2240 teams with a score of 0.81716. This corresponds to approximately the top 4.3% of ranked teams in the downloaded snapshot. The best clean single-model submission reached 0.81061 and Rank 238.

Figure 13 shows the overall public-score trajectory, while Figure 14 places our result inside the distribution of all public leaderboard scores. For course-compliance purposes, Figure 15 also includes representative Kaggle submission milestones retrieved through the Kaggle API on May 17, 2026. Semantic labels are used for readability, while the raw submission history is preserved in the exported tables.

VI. DISCUSSION

Three lessons are worth emphasizing.

First, *Spaceship Titanic* is a poor setting for chasing public score without scrutiny if the goal is an academically strong report. Because the dataset is small and widely examined, public-score shifts can reflect quirks of a particular split rather than stable behavior across samples. This concern is consistent with prior work on leaderboard reliability, which treats repeated public-score interaction as a genuine evaluation problem rather than a harmless convenience [8].

Second, the strongest clean single-model pipeline and the strongest public-scoring pipeline are not the same object. The clean CatBoost line is the better teaching and reporting anchor because it is transparent, stable, and easy to justify. The public-best XGBoost branch is the correct competition artifact because it delivered the highest team score. Separating these roles resolves the central tension between explanation and leaderboard performance.

Third, the study structure itself strengthens credibility. Repeated group-aware validation, the separation between search seeds and audit seeds, and the refusal to promote tiny gains without wider checking all made the workflow more trustworthy. For this dataset, staying consistent matters more than small visible score changes.

VII. THREATS TO VALIDITY

This project has four main validity threats.

First, the public leaderboard is only a partial view of the hidden Kaggle test set. A visible public improvement may not transfer to the private split. This is why the paper does not equate the best public rank with universal model superiority.

Second, the competition-side XGBoost branch uses choices that are more acceptable in a competition than in a rigorous scientific benchmark, especially train-plus-test harmonization and manual encoded-feature pruning. We therefore describe it explicitly as a competition-side branch rather than as the primary research conclusion.

Third, the final public-best XGBoost branch includes seed-specific competition-side selection. We therefore report it as the submitted entry rather than as a generally optimal hyperparameter setting.

Fourth, the whole analysis remains inside one competition ecosystem. Although our validation discipline is stronger than a naive train-test split, the conclusions apply specifically to *Spaceship Titanic*. The work is best read as a careful case study in tabular competition methodology.

VIII. FUTURE WORK AND CONCLUSION

If the project continued beyond the course deadline, another leaderboard-oriented adjustment would not be the first priority. More meaningful directions would include nested group-aware hyperparameter tuning, fold-wise error analysis for group-consistency failures, probability calibration, and stability checks for feature-importance explanations. A finer probability-level analysis of CatBoost–XGBoost disagreement regions could also improve interpretability.

In conclusion, the project achieved both goals, but not through a single model line. The competitive effort reached

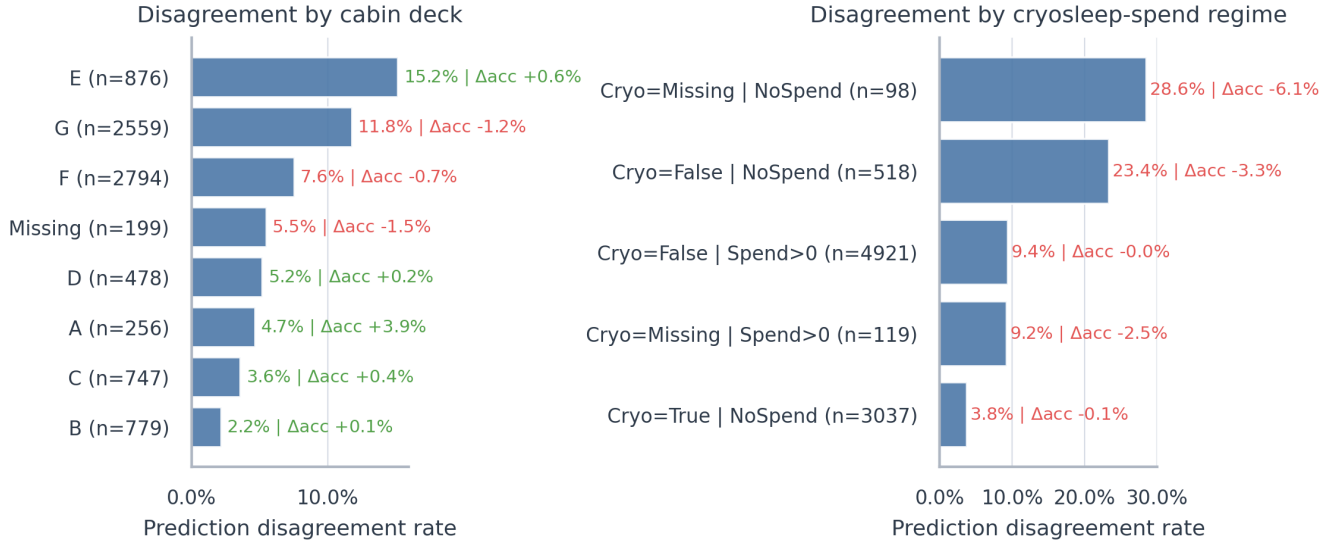


Fig. 12. Shared-audit disagreement between the clean CatBoost line and the compact XGBoost branch. Text annotations report subgroup disagreement rate and local accuracy gap (XGBoost minus CatBoost).

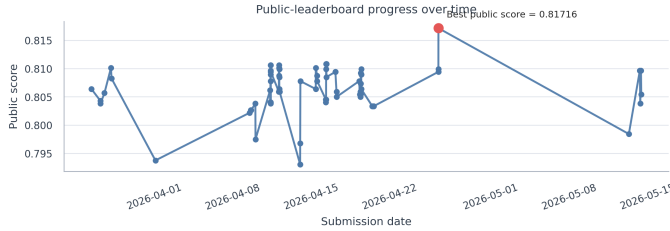


Fig. 13. Public leaderboard progress over time. The best visible submission remains the XGBoost branch submitted on April 26, 2026.

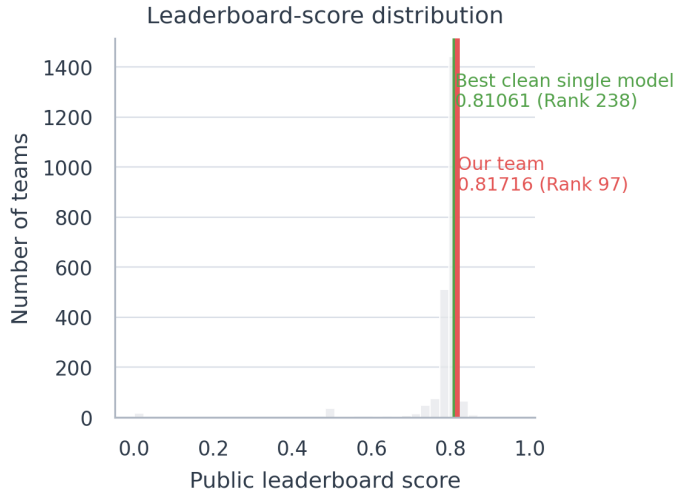


Fig. 14. Leaderboard-score distribution in the public snapshot downloaded on May 17, 2026. The team result and the best clean single-model result are highlighted.

0.81716 and Rank 97/2240 in the public leaderboard snapshot dated May 17, 2026. The methodological effort established a reproducible validation-first workflow, benchmarked five mod-

els, documented computational trade-offs, and showed that local validation and public leaderboard behavior can diverge in both directions. For a widely examined Kaggle dataset such as *Spaceship Titanic*, the central lesson is that trustworthy results come from systematic checks and step-by-step testing, not from assuming that a higher public score always means deeper insight.

IX. CODE AVAILABILITY AND CONTRIBUTION STATEMENT

A. Code Availability

The final IEEE paper, generated figures, tables, reproducible notebook, and code are organized in the submitted repository:

<https://github.com/Dylan4X/spaceship-titanic-mlworkshop>

B. Contribution Statement

Dingrong Xue contributed 34% of the project work, including Kaggle leaderboard experimentation, validation design, model benchmarking, final report writing, experimental refinement, and reproducible code preparation. Fan Xie contributed 33%, including Kaggle leaderboard experimentation, competition-side XGBoost exploration, hyperparameter search, result discussion, and presentation slide preparation. Shijun Cheng contributed 33%, including exploratory data analysis, feature engineering support, interpretation of dataset patterns, visualization preparation, and presentation slide preparation. All members participated in final project review.

APPENDIX A REQUIREMENT TRACEABILITY

Table X maps the course report requirements to concrete evidence in this paper.

Milestone	Submission time	Public score	Role
Clean CatBoost exact line	2026-04-11 09:36	0.81061	Clean anchor
Flaykaer clean reproduction	2026-04-12 16:09	0.80991	Clean reproduction
High-CV CatBoost counterexample	2026-04-12 17:34	0.80593	Validation counterexample
XGB-assisted stack	2026-04-16 11:57	0.81084	Competition-side hybrid
Final public-best XGB branch	2026-04-26 08:10	0.81716	Public-best branch
Late blend attempt	2026-05-14 09:37	0.80967	Late check

Fig. 15. Representative Kaggle submission milestones retrieved on May 17, 2026. Semantic labels are used in place of raw file names; the best visible score is the final public-best XGBoost branch at 0.81716.

TABLE X
TRACEABILITY FROM COURSE REQUIREMENTS TO EVIDENCE IN THE FINAL PAPER

Course requirement	Evidence in this paper
Abstract	Abstract section on page 1 summarizes the task, methods, and final outcomes.
Introduction	Section I explains the competition setting and the distinction between leaderboard outcome and scientific claim.
Related work / existing techniques	Section II discusses boosting models, validation practice, and representative Kaggle guides.
Methodology	Section IV documents preprocessing, feature engineering, validation design, and the competition-side XGBoost branch.
Experimental study and results analysis	Section V contains the unified benchmark, CV-public alignment analysis, XGBoost branch reconstruction, and final ranking discussion.
Future work and conclusion	Section VIII provides both next steps and final conclusions.
References	The bibliography cites the competition page, model papers, Optuna, and representative Kaggle notebooks.
Contribution statement	Section IX contains a member-level contribution statement with named authors and workload percentages.
Final Kaggle ranking	Section V-E and Figures 13–14 report Rank 97/2240 with public score 0.81716 in the May 17, 2026 snapshot.
GitHub link of the code	Section IX-A provides the repository URL field for the submitted code.
Screenshot(s) of Kaggle submission log(s)	Figure 15 provides a cleaned submission-log snapshot with representative milestones retrieved on May 17, 2026.

APPENDIX B MILESTONE SUBMISSION RECORD

Table XI records the milestone submissions that matter most to the final narrative.

This appendix is included for two practical reasons. First, the course requires evidence of the final Kaggle workflow rather than only the final score. Second, the main body of the paper repeatedly distinguishes between clean scientific anchors and competition-side artifacts; the milestone list makes that distinction concrete in submission-history terms.

Read together with Figure 13, this appendix clarifies the project chronology. The April 26, 2026 XGBoost branch remained the best visible public result through the final snapshot on May 17, 2026, while the cleaner CatBoost line remained the report’s main scientific anchor. That dual interpretation is central to the paper and is therefore documented explicitly here.

REFERENCES

- [1] Kaggle, “Spaceship Titanic,” online competition page. Available: <https://www.kaggle.com/competitions/spaceship-titanic/overview>
- [2] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, 2017.
- [4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “CatBoost: unbiased boosting with categorical features,” in *Advances in Neural Information Processing Systems*, 2018.
- [5] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [6] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [7] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.
- [8] A. Blum and M. Hardt, “The Ladder: A reliable leaderboard for machine learning competitions,” in *Proc. 32nd Int. Conf. Machine Learning*, 2015, pp. 1006–1014.
- [9] S. Cortinhas, “Spaceship Titanic: A complete guide,” Kaggle notebook, 2022. Available: <https://www.kaggle.com/code/samuelcortinhas/spaceship-titanic-a-complete-guide>
- [10] Arunklenin, “Space Titanic — EDA — Advanced Feature Engineering,” Kaggle notebook, 2024. Available: <https://www.kaggle.com/code/arunklenin/space-titanic-eda-advanced-feature-engineering>

TABLE XI
MILESTONE SUBMISSION RECORD USED IN THE FINAL ANALYSIS

Date	Milestone label	Public score	Pipeline type	Why it matters
2026-04-11	Clean CatBoost single model	0.81061	Clean single model	Best clean single-model anchor used for most of the report narrative.
2026-04-12	Clean CatBoost reproduction	0.80991	Clean reproduction	Transparent group-aware CatBoost reproduction that remained interpretable but did not beat the clean anchor.
2026-04-12	High-CV CatBoost candidate	0.80593	High-CV clean candidate	Strong local-CV counterexample showing that public and local metrics can diverge sharply.
2026-04-16	XGB-assisted stack	0.81084	Competition-side hybrid	Demonstrates that XGBoost became useful before the final public-best branch.
2026-04-26	Final public-best XGB branch	0.81716	Competition-side compact branch	Highest public score in the team history and the endpoint of the XGBoost exploration path.
2026-05-14	Late blend attempt	0.80967	Late ensemble check	Confirms that later submissions did not surpass the April 26, 2026 public best.